关于进程保护的探究

渗透测试 (https://forum.butian.net/topic/47)

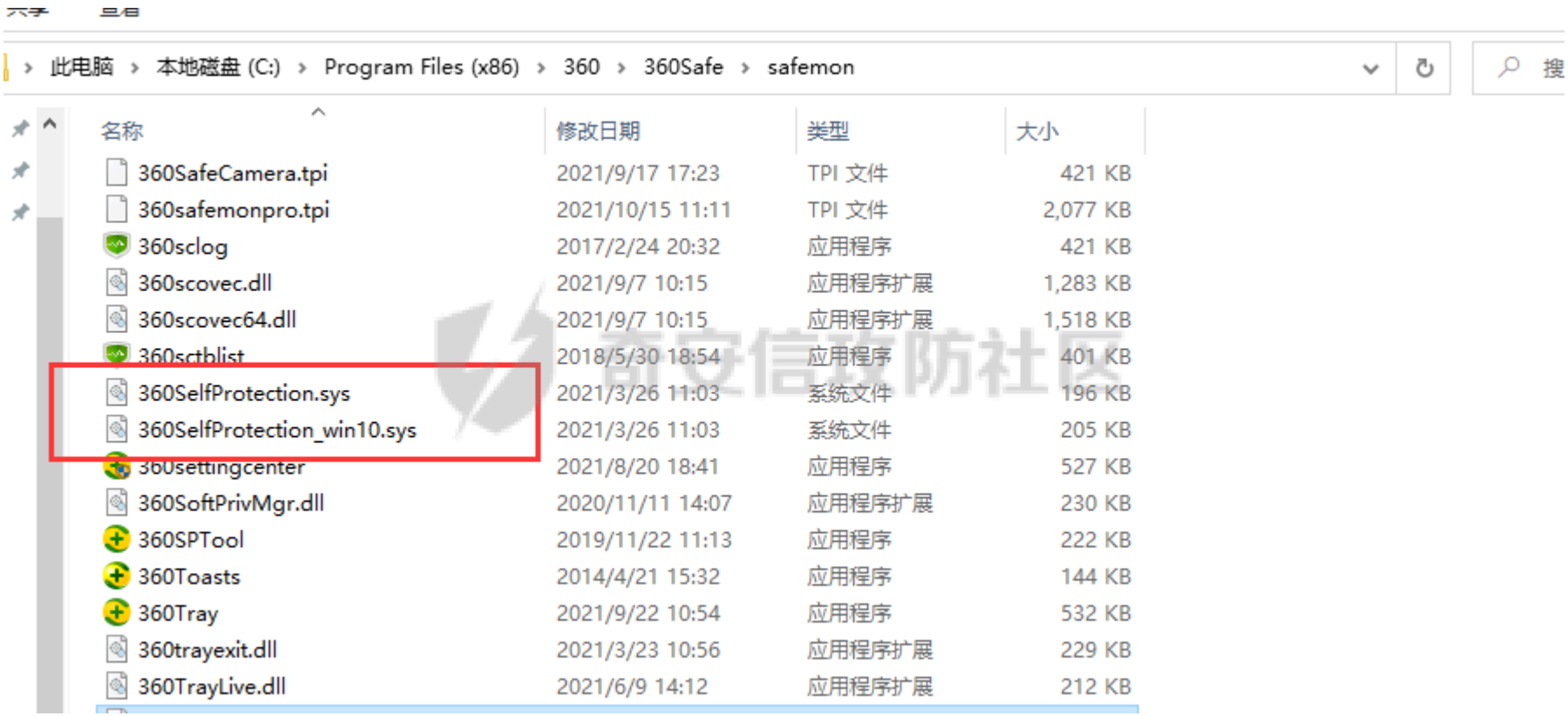
最近在研究某数字杀软的时候看到有个配置选项：

前言

最近在研究某数字杀软的时候看到有个配置选项：

(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-8ef9216d5dae8c8f13709b821d684450f237ac94.png)

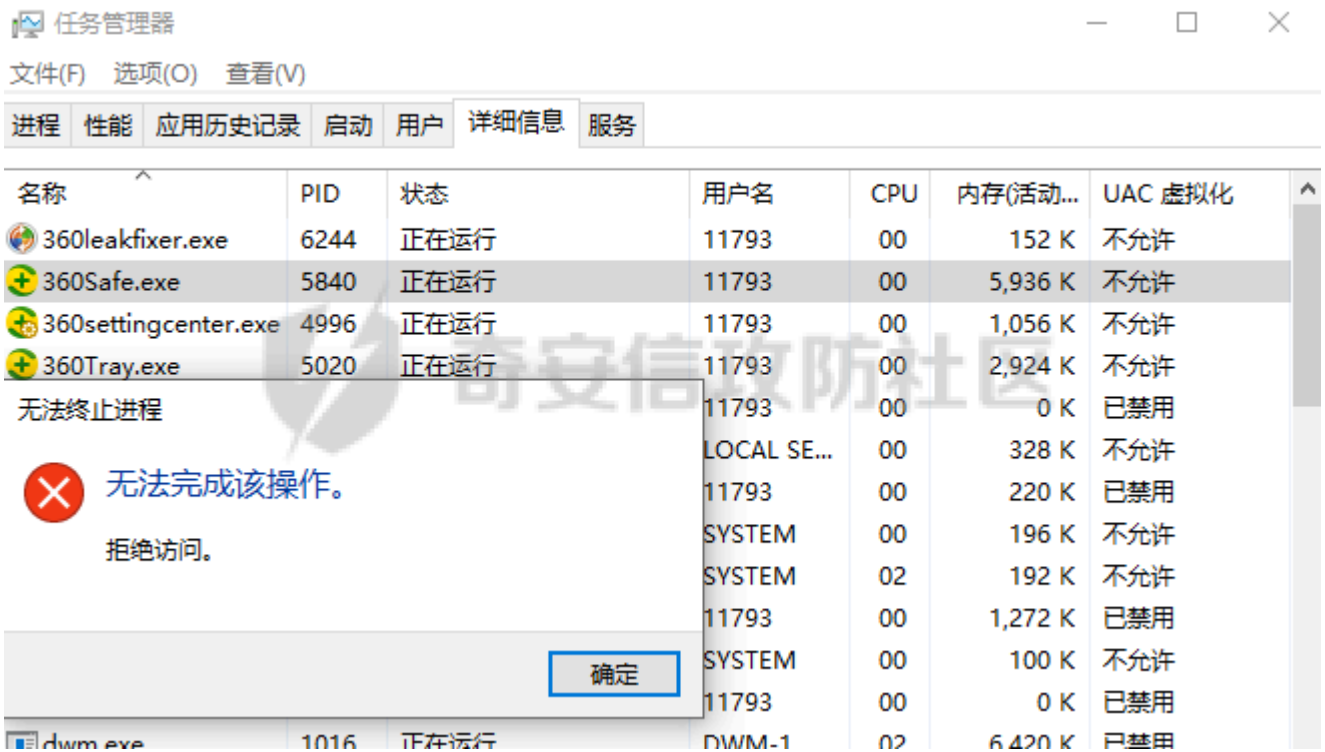
这个自我保护实际上是加载360SelfProtection.sys驱动(看这名字应该还有360SelfProtection_win10.sys文件)，在0环通过hook等手段保护注册表项，重要进程进程等。



(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-9290efc1d25b098c3a89424fe4955dd5d9d61d35.png)

比如这里要结束某核心进程，会显示无法结束，拒绝访问。

(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-467ae61b31ae241b4af62ac1307c047e8e2df318.png)



(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-a33f76fe33dab7701bf02df2156801db5525da91.png)

这个并不是说权限不够的问题，即便是system权限也不行。而是由于在底层，杀死进程的API已经被hook了，这里应该是内核hook，当想要结束的进程为核心进程时，就直接返回一个无法终止进程的弹窗。本文就如何实现一个进程保护功能进行探究，驱动就不写了，就写一个用户层的。

实现原理

windows提供了一个可以杀死其他进程的API：TerminateProcess。如果是结束本身进程为：ExitProcess。

```
BOOL TerminateProcess(
  [in] HANDLE hProcess,
  [in] UINT   uExitCode
);
```

通过命令taskkill或者通过任务管理器等GUI工具去结束进程，本质上都是调用的TerminateProcess，有些可能是调用更为底层的ZwTerminateProcess，但是本质都差不多，只是看该进程用的什么API，这里为了方便就使用TerminateProcess进行演示。

通过hook TerminateProcess让执行该函数时直接返回没有权限。

获取API地址并创建新的hook函数

```
static BOOL(WINAPI* OldTerminateProcess)(
    HANDLE hProcess,
    UINT   uExitCode) = TerminateProcess;

BOOL WINAPI New_TerminateProcess(
    _In_ HANDLE hProcess,
    _In_ UINT uExitCode
) {
    unhookTerminateProcess();
    MessageBox(NULL,L"该进程受保护！", L"Access Denied",NULL);
    hookTerminateProcess();
    return false;
}
```

这里是64位的进程，要改变的硬编码是12个字节，而且不需要去算偏移。

```
48 b8 _dwNewAddress(0x1122334455667788)
ff e0
mov rax, _dwNewAddress(0x1122334455667788)
jmp rax
```

将原来函数的调转地址改为我们自己函数的跳转地址

```
void hookTerminateProcess() {
    DWORD dwTerminateProcessOldProtect;
    BYTE pData[12] = { 0x48,0xb8,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0xFF,0xE0 };
    ULONGLONG ullNewFuncAddr = (ULONGLONG)New_TerminateProcess;
    //保存本来的字节
    ::RtlCopyMemory(g_OldTerminateProcess, OldTerminateProcess, sizeof(pData));
    //更改权限
    VirtualProtect(OldTerminateProcess, 12, PAGE_EXECUTE_READWRITE, &dwTerminateProcessOldProtect);
    //更改原来的机器码，改为我们自己函数的地址
    ::RtlCopyMemory(&pData[2], &ullNewFuncAddr, sizeof(ullNewFuncAddr));
    ::RtlCopyMemory(OldTerminateProcess, pData, sizeof(pData));
    //恢复属性
    VirtualProtect(OldTerminateProcess, 12, dwTerminateProcessOldProtect, &dwTerminateProcessOldProtect);
}
```

恢复硬编码，还原调转地址。

```
void unhookTerminateProcess() {
    DWORD dwOldProtect = NULL;
    //改为可写属性
    VirtualProtect(OldTerminateProcess, 12, PAGE_EXECUTE_READWRITE, &dwOldProtect);
    //还原原来的硬编码
    RtlCopyMemory(OldTerminateProcess, g_OldTerminateProcess, sizeof(g_OldTerminateProcess));
    //还原属性
    VirtualProtect(OldTerminateProcess, 12, dwOldProtect, &dwOldProtect);
}
```

最方便的就是写成一个dll，这样注入到任务管理器并起一个线程跑就行了。

```
BOOL APIENTRY DllMain( HMODULE hModule,
                       DWORD  ul_reason_for_call,
                       LPVOID lpReserved
                     )
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        hookTerminateProcess();
        break;
    case DLL_THREAD_ATTACH:
        break;
    case DLL_THREAD_DETACH:
        break;
    case DLL_PROCESS_DETACH:
        unhookTerminateProcess();
        break;
    }
    return TRUE;
}
```

注入这里也是为了练手，稍微写了一个注入程序，为了以后的方法写成了突破session 0的注入。

```
#include <iostream>
#include <windows.h>
#include "tchar.h"
#include <TlHelp32.h>

using namespace std;

BOOL EnbalePrivileges(HANDLE hProcess, LPCWSTR pszPrivilegesName)
{
    HANDLE hToken = NULL;
    LUID luidValue = { 0 };
    TOKEN_PRIVILEGES tokenPrivileges = { 0 };
    BOOL bRet = FALSE;
    DWORD dwRet = 0;
    // 打开进程令牌并获取进程令牌句柄
    bRet = ::OpenProcessToken(hProcess, TOKEN_ADJUST_PRIVILEGES, &hToken);
    if (FALSE == bRet)
    {
        printf("[!] Open CurrentProcessToken Error,Error is:%d\n",GetLastError());
        return FALSE;
    }
    else
    {
        printf("[*] Open CurrentProcessToken Successfully!,TokenHandle is:%d\n", hToken);
    }
    // 获取本地系统的 pszPrivilegesName 特权的LUID值
    bRet = ::LookupPrivilegeValue(NULL, pszPrivilegesName, &luidValue);
    if (FALSE == bRet)
    {
        printf("[!] LookupPrivilegeValue Error,Error is:%d\n", GetLastError());
        return FALSE;
    }
    else
    {
        printf("[*] LookupPrivilegeValue Successfully!\n");
```

```
    }
    // 设置提升权限信息
    tokenPrivileges.PrivilegeCount = 1;
    tokenPrivileges.Privileges[0].Luid = luidValue;
    tokenPrivileges.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;
    // 提升进程令牌访问权限
    bRet = ::AdjustTokenPrivileges(hToken, FALSE, &tokenPrivileges, 0, NULL, NULL);
    if (FALSE == bRet)
    {
        printf("[!] AdjustTokenPrivileges Error,Error is:%d\n", GetLastError());
        return FALSE;
    }
    else
    {
        // 根据错误码判断是否特权都设置成功
        dwRet = ::GetLastError();
        if (ERROR_SUCCESS == dwRet)
        {
            printf("[√] ALL_ASSIGNED!\n");
            return TRUE;
        }
        else if (ERROR_NOT_ALL_ASSIGNED == dwRet)
        {
            printf("[!] ERROR:NOT_ALL_ASSIGNED,Error is %d\n", dwRet);
            return FALSE;
        }
    }
    return FALSE;
}
DWORD EnumModules(DWORD hPid, LPCSTR hMoudlePath)
{
    WCHAR szBuffer[MAX_PATH] = { 0 };
    mbstowcs(szBuffer, hMoudlePath, MAX_PATH);
    //通过pid列出所有的Modules
    HANDLE hModuleSnap = INVALID_HANDLE_VALUE;
    MODULEENTRY32    me32;
```

```
    //给进程所引用的模块信息设定一个快照
    hModuleSnap = CreateToolhelp32Snapshot(TH32CS_SNAPMODULE, hPid);
    if (hModuleSnap == INVALID_HANDLE_VALUE)
    {
        printf("[!] Error:Enum modules failed to detect if there is an injected DLL module,error is %d\n" ,GetLastErro
    }
    me32.dwSize = sizeof(MODULEENTRY32);
    if (!Module32First(hModuleSnap, &me32))
    {
        printf("[!] Enum Error!\n");
        CloseHandle(hModuleSnap);
    }
    do
    {
        if(!memcmp(me32.szExePath, szBuffer,MAX_PATH))
            return 1;
    } while (Module32Next(hModuleSnap, &me32));
    CloseHandle(hModuleSnap);
    return 0;
}
DWORD _InjectThread(DWORD _Pid, LPCSTR psDllPath)
{
    FILE* fp;
    fp = fopen(psDllPath, "r");
    if (!fp)
    {
        printf("[!] Error:DLL path not found\nPlease check that your path is correct or absolute\n");
        return FALSE;
    }
    fclose(fp);
    printf("*****************************************************************************\n");
    HANDLE hprocess = NULL;
    HANDLE hThread = NULL;
    DWORD _SIZE = 0;
    LPVOID pAlloc = NULL;
```

```
FARPROC pThreadFunction = NULL;
DWORD ZwRet = 0;
hprocess = ::OpenProcess(PROCESS_ALL_ACCESS, FALSE, _Pid);
if (hprocess == NULL)
{
    printf("[!] OpenProcess Error,Error is:%d\n", GetLastError());
    return FALSE;
}
else
{
    printf("[*] OpenProcess Successfully!\n");
}
_SIZE = strlen(psDllPath)+1;
pAlloc = ::VirtualAllocEx(hprocess, NULL, _SIZE, MEM_COMMIT, PAGE_READWRITE);
if (pAlloc == NULL)
{
    printf("[!] VirtualAllocEx Error,Error is:%d\n", GetLastError());
    return FALSE;
}
else
{
    printf("[*] VirtualAllocEx Successfully!\n");
}
BOOL x = ::WriteProcessMemory(hprocess, pAlloc, psDllPath, _SIZE, NULL);
if (FALSE == x)
{
    printf("[!] WriteMemory Error,Error is:%d\n", GetLastError());
    return FALSE;
}
else
{
    printf("[*] WriteMemory Successfully!\n");
}

HMODULE hNtdll = LoadLibrary(L"ntdll.dll");
if (hNtdll == NULL)
```

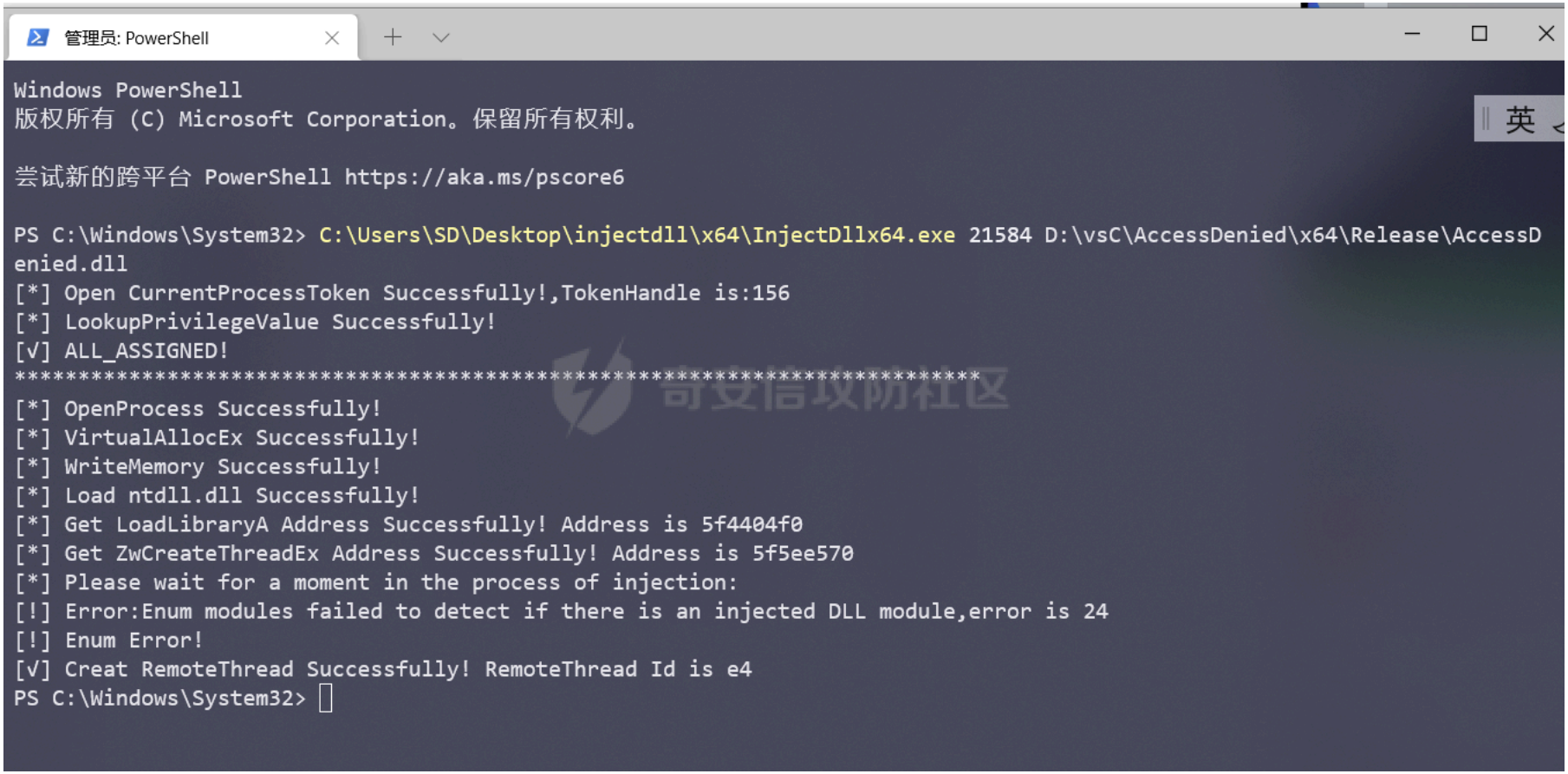
```
    {
        printf("[!] LoadNTdll Error,Error is:%d\n", GetLastError());
        return FALSE;
    }
    else
    {
        printf("[*] Load ntdll.dll Successfully!\n");
    }
    pThreadFunction = ::GetProcAddress(::GetModuleHandle(L"kernel32.dll"), "LoadLibraryA");
    if (pThreadFunction == NULL)
    {
        printf("[!] Get LoadLibraryA Address Error,Error is:%d\n", GetLastError());
        return FALSE;
    }
    else
    {
        printf("[*] Get LoadLibraryA Address Successfully! Address is %x\n", pThreadFunction);
    }
#ifdef _WIN64
    typedef DWORD(WINAPI* typedef_ZwCreateThreadEx)(
        PHANDLE ThreadHandle,
        ACCESS_MASK DesiredAccess,
        LPVOID ObjectAttributes,
        HANDLE ProcessHandle,
        LPTHREAD_START_ROUTINE lpStartAddress,
        LPVOID lpParameter,
        ULONG CreateThreadFlags,
        SIZE_T ZeroBits,
        SIZE_T StackSize,
        SIZE_T MaximumStackSize,
        LPVOID pUnkown
        );
#else
    typedef DWORD(WINAPI* typedef_ZwCreateThreadEx)(
        PHANDLE ThreadHandle,
        ACCESS_MASK DesiredAccess,
```

```
        LPVOID ObjectAttributes,
        HANDLE ProcessHandle,
        LPTHREAD_START_ROUTINE lpStartAddress,
        LPVOID lpParameter,
        BOOL CreateSuspended,
        DWORD dwStackSize,
        DWORD dw1,
        DWORD dw2,
        LPVOID pUnkown
        );
#endif
    typedef_ZwCreateThreadEx ZwCreateThreadEx = NULL;
    ZwCreateThreadEx = (typedef_ZwCreateThreadEx)::GetProcAddress(hNtdll, "ZwCreateThreadEx");

    if (ZwCreateThreadEx == NULL)
    {
        printf("[!] Get ZwCreateThreadEx Address Error,Error is:%d\n", GetLastError());
        return FALSE;
    }
    else
    {
        printf("[*] Get ZwCreateThreadEx Address Successfully! Address is %x\n", ZwCreateThreadEx);
    }
    HANDLE hRemoteThread;
    ZwRet = ZwCreateThreadEx(&hRemoteThread, PROCESS_ALL_ACCESS, NULL, hprocess,
        (LPTHREAD_START_ROUTINE)pThreadFunction, pAlloc, 0, 0, 0, 0, NULL);

    if (hRemoteThread == NULL)
    {
        printf("[!] Creat RemoteThread Error,Error is:%d\n", GetLastError());
        CloseHandle(hprocess);
        return FALSE;
    }

    printf("[*] Please wait for a moment in the process of injection:\n");
    for(int m = 0;m<5;m++)
```

```
    {
        if (EnumModules(_Pid, psDllPath))
        {
            printf("[√] Creat RemoteThread Successfully! RemoteThread Id is %x\n", hRemoteThread);
            VirtualFreeEx(hprocess, pAlloc, 0, MEM_RELEASE);
            CloseHandle(hRemoteThread);
            CloseHandle(hprocess);
            FreeLibrary(hNtdll);
            return TRUE;
        }
        Sleep(2000);
    }
    printf("[!] DLL injection failed!\nNotice:Please check that your path is absolute or correct\n");

    VirtualFreeEx(hprocess, pAlloc, 0, MEM_RELEASE);
    CloseHandle(hRemoteThread);
    CloseHandle(hprocess);
    FreeLibrary(hNtdll);
    return FALSE;
}
int main(int argc, char* argv[])
{
    if (argc == 3)
    {
        EnbalePrivileges(GetCurrentProcess(), SE_DEBUG_NAME);
        DWORD dwPid;
        sscanf(argv[1],"%d", &dwPid);
        _InjectThread(dwPid, argv[2]);
        return 1;
    }
    else
    {
        printf("[!] You passed in the wrong number of parameters!Please pass in two parameters.\n");
        printf("[!] Notice:\n[!] The first parameter is the PID of the target process\n[!] The second parameter is the
        return 0;
```

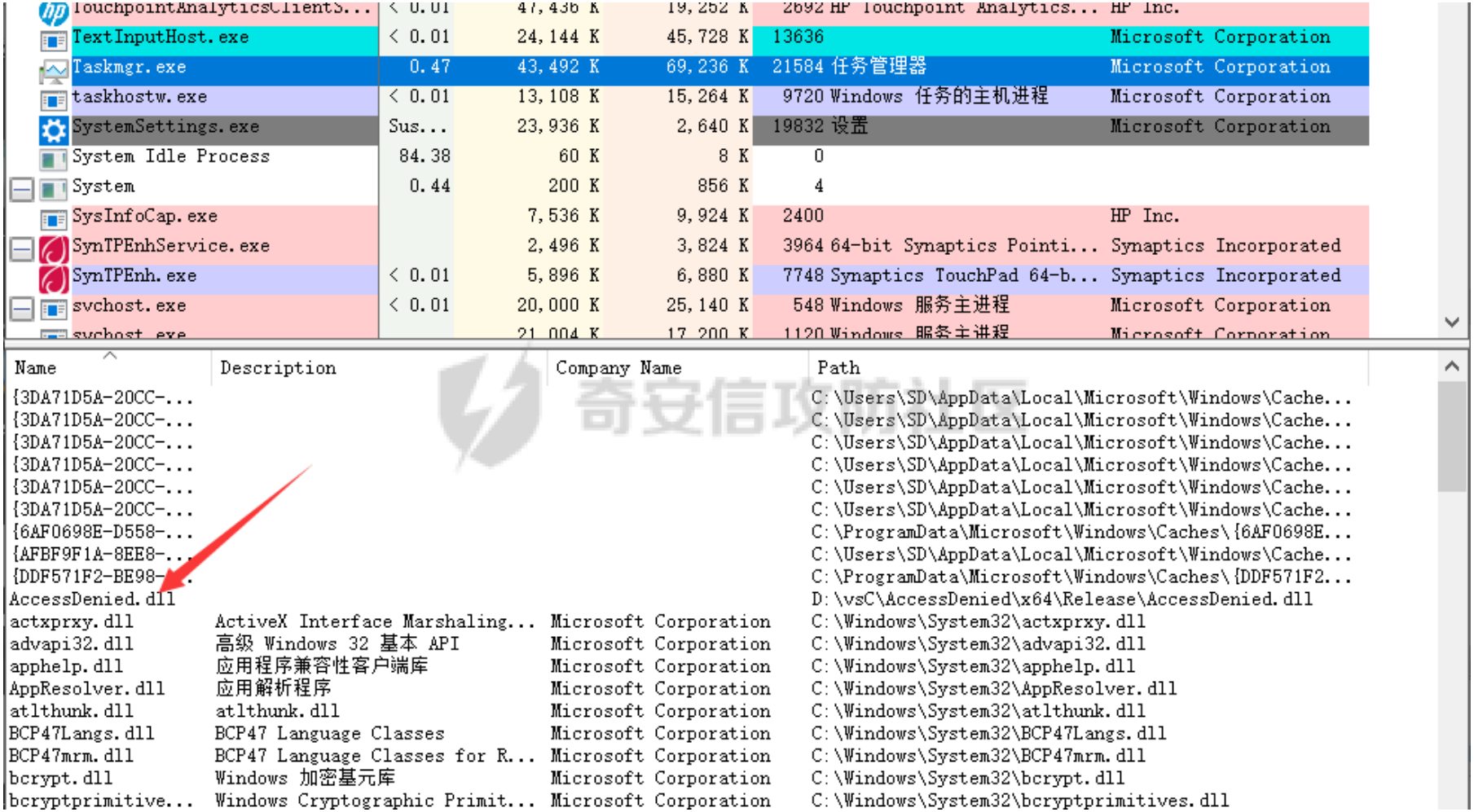
```
    }
}
```

直接注入，看到线程创建成功就表示已经成功了。



(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-d63db9863c4d9c717519fbf5c0c952bae2436fd4.png)

用procexp也可以看一眼。

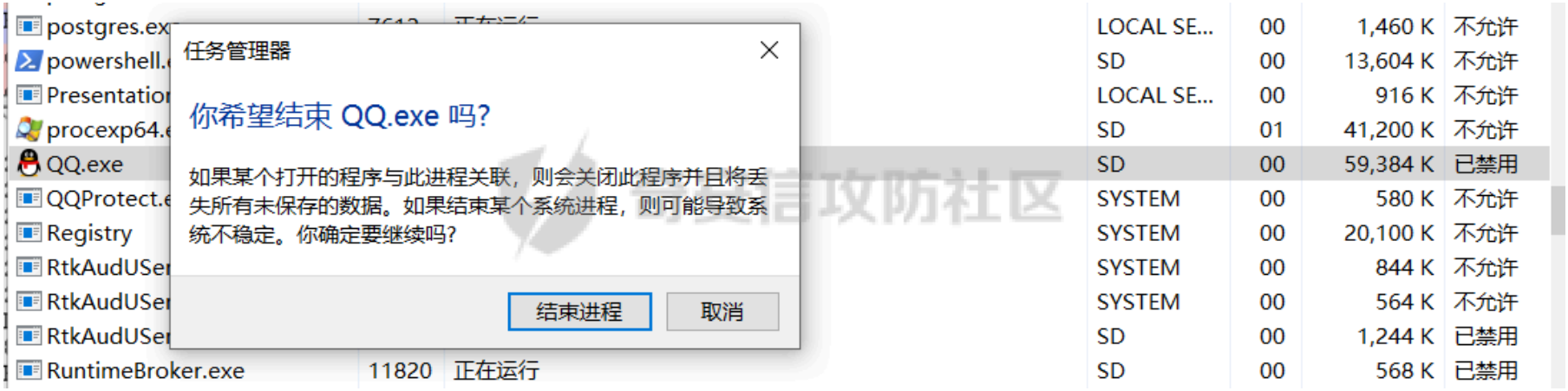


TouchpointAnalyticsClientS...	< 0.01	47,436 K	19,252 K	2692 HP Touchpoint Analytics...	HP Inc.
TextInputHost.exe	< 0.01	24,144 K	45,728 K	13636	Microsoft Corporation
Taskmgr.exe	0.47	43,492 K	69,236 K	21584 任务管理器	Microsoft Corporation
taskhostw.exe	< 0.01	13,108 K	15,264 K	9720 Windows 任务的主机进程	Microsoft Corporation
SystemSettings.exe	Sus...	23,936 K	2,640 K	19832 设置	Microsoft Corporation
System Idle Process	84.38	60 K	8 K	0	
System	0.44	200 K	856 K	4	
SysInfoCap.exe		7,536 K	9,924 K	2400	HP Inc.
SynTPEnhService.exe		2,496 K	3,824 K	3964 64-bit Synaptics Pointi...	Synaptics Incorporated
SynTPEnh.exe	< 0.01	5,896 K	6,880 K	7748 Synaptics TouchPad 64-b...	Synaptics Incorporated
svchost.exe	< 0.01	20,000 K	25,140 K	548 Windows 服务主进程	Microsoft Corporation
svchost.exe		21,004 K	17,200 K	1120 Windows 服务主进程	Microsoft Corporation

Name	Description	Company Name	Path
{3DA71D5A-20CC-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{3DA71D5A-20CC-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{3DA71D5A-20CC-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{3DA71D5A-20CC-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{3DA71D5A-20CC-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{3DA71D5A-20CC-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{6AF0698E-D558-...			C:\ProgramData\Microsoft\Windows\Caches\{6AF0698E...
{AFBF9F1A-8EE8-...			C:\Users\SD\AppData\Local\Microsoft\Windows\Cache...
{DDF571F2-BE98-...			C:\ProgramData\Microsoft\Windows\Caches\{DDF571F2...
AccessDenied.dll			D:\vsC\AccessDenied\x64\Release\AccessDenied.dll
actxprxy.dll	ActiveX Interface Marshaling...	Microsoft Corporation	C:\Windows\System32\actxprxy.dll
advapi32.dll	高级 Windows 32 基本 API	Microsoft Corporation	C:\Windows\System32\advapi32.dll
apphelp.dll	应用程序兼容性客户端库	Microsoft Corporation	C:\Windows\System32\apphelp.dll
AppResolver.dll	应用解析程序	Microsoft Corporation	C:\Windows\System32\AppResolver.dll
atlthunk.dll	atlthunk.dll	Microsoft Corporation	C:\Windows\System32\atlthunk.dll
BCP47Langs.dll	BCP47 Language Classes	Microsoft Corporation	C:\Windows\System32\BCP47Langs.dll
BCP47mrm.dll	BCP47 Language Classes for R...	Microsoft Corporation	C:\Windows\System32\BCP47mrm.dll
bcrypt.dll	Windows 加密基元库	Microsoft Corporation	C:\Windows\System32\bcrypt.dll
bcryptprimitive...	Windows Cryptographic Primit...	Microsoft Corporation	C:\Windows\System32\bcryptprimitives.dll

(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-d6701f406436c1cef18d472e3743946e7c02a6a6.png)

比如现在我们想要结束QQ进程。



(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-8a7a4e54e23e5dccf520e8316ddde70571377059.png)

会发现无法结束。

(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-61c713a5069606cb8278c858c5252fc58f0e0905.png)

这里由于还没有写判断规则，这个任务管理器目前无法结束任何进程，也就是所有进程都无法结束，这显然用户体验是不好的。

那么如果要选择性保护进程，又应该怎么做呢。注意TerminateProcess的第一个参数，传入的是一个句柄，这个句柄需要从openprocess的返回值获得，所以我们还需要知道打开进程的句柄。

> 同一进程多次使用openprocess获取的句柄是不一样的。

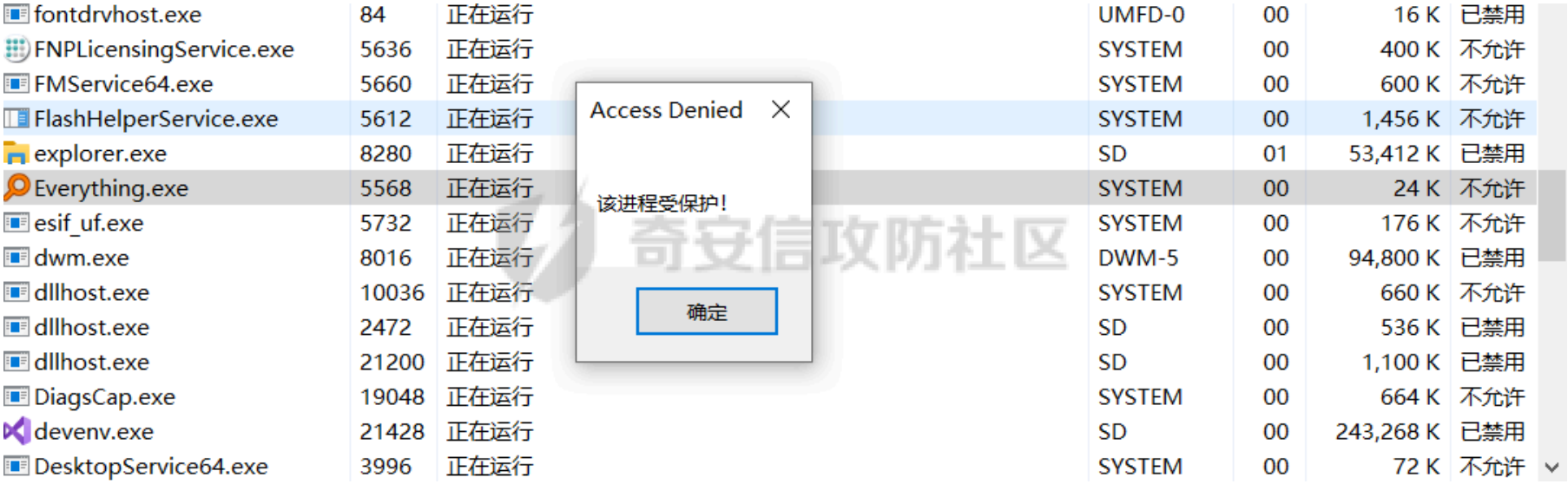
inline hook的稳定性还是差了点，很容易让进程崩溃。根据实验：任务管理器会不断地调用openprocess这个api，不管有没有操作都会一直调用。这里就用微软更加稳定的detours库。

```
void Hook()
{
    DetourTransactionBegin();
    DetourUpdateThread(GetCurrentThread());
    // 参数一是原函数地址，参数二是新函数地址
    DetourAttach((PVOID*)&OldOpenProcess, New_OpenProcess);
    DetourAttach((PVOID*)&OldTerminateProcess, New_TerminateProcess);
    DetourTransactionCommit();
}

void UnHook()
{
    DetourTransactionBegin();
    DetourUpdateThread(GetCurrentThread());
    // 和Hook完全一样，不同的只是将DetourAttach换成DetourDetach
    DetourDetach((PVOID*)&OldTerminateProcess, New_TerminateProcess);
    DetourDetach((PVOID*)&OldOpenProcess, New_OpenProcess);
    DetourTransactionCommit();
}
```

我们自己的函数如下编写，就是做一些简单的判断，比如这里要保护的进程id为5568：

```
HANDLE g_handle = NULL;
HANDLE WINAPI New_OpenProcess(DWORD dwDesiredAccess,BOOL bInheritHandle,DWORD dwProcessId) {

    if (dwProcessId == 5568) {
        g_handle = OldOpenProcess(dwDesiredAccess, bInheritHandle, dwProcessId);
        return g_handle;
    }
    else
    {
        HANDLE hHandle = OldOpenProcess(dwDesiredAccess, bInheritHandle, dwProcessId);
        return hHandle;
    }
}

BOOL WINAPI New_TerminateProcess(
    _In_ HANDLE hProcess,
    _In_ UINT uExitCode
) {
    if (g_handle == hProcess)
    {
        MessageBox(NULL, L"该进程受保护！", L"Access Denied", NULL);
        g_handle = NULL;
        return false;
    }
    else
    {
        OldTerminateProcess(hProcess, uExitCode);
        return true;
    }

}
```

当我尝试终止5568进程时会拒绝执行。



fontdrvhost.exe	84	正在运行	UMFD-0	00	16 K	已禁用
FNPLicensingService.exe	5636	正在运行	SYSTEM	00	400 K	不允许
FMService64.exe	5660	正在运行	SYSTEM	00	600 K	不允许
FlashHelperService.exe	5612	正在运行	SYSTEM	00	1,456 K	不允许
explorer.exe	8280	正在运行	SD	01	53,412 K	已禁用
Everything.exe	5568	正在运行	SYSTEM	00	24 K	不允许
esif_uf.exe	5732	正在运行	SYSTEM	00	176 K	不允许
dwm.exe	8016	正在运行	DWM-5	00	94,800 K	已禁用
dllhost.exe	10036	正在运行	SYSTEM	00	660 K	不允许
dllhost.exe	2472	正在运行	SD	00	536 K	已禁用
dllhost.exe	21200	正在运行	SD	00	1,100 K	已禁用
DiagsCap.exe	19048	正在运行	SYSTEM	00	664 K	不允许
devenv.exe	21428	正在运行	SD	00	243,268 K	已禁用
DesktopService64.exe	3996	正在运行	SYSTEM	00	72 K	不允许

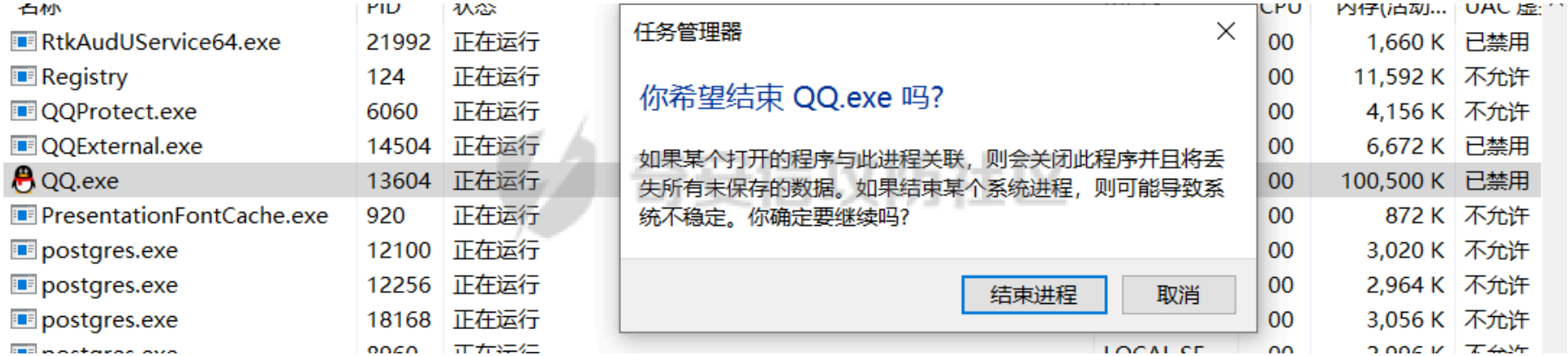
(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-87b72250b5df0e62af2c805bf805d7501d9351c3.png)

尝试终止其他的程序则成功终止。比如这里有个QQ进程。

RtkAudUService64.exe	4660	正在运行	SYSTEM	00	656 K	不允许
RtkAudUService64.exe	21992	正在运行	SD	00	1,628 K	已禁用
Registry	124	正在运行	SYSTEM	00	11,544 K	不允许
QQProtect.exe	6060	正在运行	SYSTEM	00	4,024 K	不允许
QQ.exe	13604	正在运行	SD	02	67,828 K	已禁用
PresentationFontCache.exe	920	正在运行	LOCAL SE...	00	872 K	不允许
postgres.exe	7620	正在运行	LOCAL SE...	00	688 K	不允许
postgres.exe	8448	正在运行	LOCAL SE...	00	344 K	不允许
postgres.exe	8492	正在运行	LOCAL SE...	00	92 K	不允许
postgres.exe	8560	正在运行	LOCAL SE...	00	112 K	不允许
postgres.exe	8640	正在运行	LOCAL SE...	00	444 K	不允许
postgres.exe	8720	正在运行	LOCAL SE...	00	348 K	不允许

(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-51cb4333cf6e232a3411a7b6893d5e8786ef8b05.png)

可直接终止进程（这里本来有个gif，社区不能上传）



(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-fdade6afb069ee237aa0faf267acd6446f8b7c6c.png)

(https://cdn-yg-zzbm.yun.qianxin.com/attack-forum/2021/10/attach-183ffcc1811f4b6ad20f9374acd7d9f5ad73d09b.png)

发表于 2021-11-01 15:51:53 阅读 (8891) 分类：安全开发 (https://forum.butian.net/community/develop)

站长统计 (https://www.cnzz.com/stat/website.php?web_id=1279782571)